

Tag 4

Erste Skripte

Befehle in eine Datei schreiben und starten. Teil 1: Shebang, Kommentare, `chmod +x`, `./`, `echo` und `sleep`. Teil 2: Variablen, Variablen vom System, `$( )` und `read`.

Warum das wichtig ist

Bis jetzt hast du jeden Befehl einzeln getippt. Das ist in Ordnung, wenn du ihn nur einmal brauchst.

Was aber, wenn du dieselben zehn Befehle jeden Montag brauchst? Dann schreibst du sie einmal in eine Datei. Danach startest du nur noch die Datei.

So eine Datei heißt **Skript**. Admins sparen mit Skripten jeden Tag viel Zeit. Der zweite Teil der Klausur ist auch ein Skript.

Teil 1: Das erste Skript bis zur großen Pause

Was ist ein Skript?

Ein Skript ist eine Textdatei mit Befehlen. Die Shell liest die Datei von oben nach unten. Sie führt jede Zeile aus. Genau wie beim Tippen im Terminal.

Skripte enden meistens auf `.sh`. Das ist keine Pflicht. Linux schaut nicht auf die Endung. Die Endung ist nur für Menschen: Man sieht sofort, dass es ein Skript ist.

Ein Skript braucht drei Dinge:

1. Die erste Zeile `#!/bin/bash`. Sie nennt die Shell.
2. Das Recht zum Ausführen.
3. Den Start mit `./` davor.

Das erste Skript

Öffne eine neue Datei:

```
nano hello.sh
```

Schreibe diese drei Zeilen hinein:

```
#!/bin/bash
# Mein erstes Skript
echo "Willkommen im Linux-Kurs!"
```

Speichere mit Strg+O und Enter. Beende mit Strg+X.

Die erste Zeile `#!/bin/bash` heißt **Shebang**. Sie sagt dem System: Führe diese Datei mit dem Programm `/bin/bash` aus. Sie muss ganz oben stehen, ohne Leerzeichen davor. Die Zeichen `#!` gibt es seit 1980. Dennis Ritchie hat sie bei Bell Labs eingeführt. Er ist einer der Erfinder von Unix.

Ist der Shebang Pflicht? Nein. Ohne ihn startet die Bash das Skript meistens trotzdem. Andere Programme melden dann aber `Exec format error`. Darum schreibst du ihn immer.

Bash heißt "Bourne Again Shell". Brian Fox hat sie 1989 für das GNU-Projekt veröffentlicht. Der Name klingt wie "born again", auf Deutsch "wiedergeboren". Er erinnert an die ältere Bourne Shell von 1979.

Bei Ubuntu gibt es auch `/bin/sh`. Das ist seit 2006 eine andere Shell. Sie heißt Dash. Sie startet schneller, kann aber weniger. Darum schreiben wir immer `/bin/bash`.

Zeilen, die mit `#` beginnen, sind **Kommentare**. Die Shell überspringt sie. Sie sind für Menschen da. Schreibe oben in jedes Skript einen Kommentar, was es tut. Der Shebang beginnt auch mit `#`. Er ist trotzdem kein Kommentar. Das System liest ihn, bevor die Bash startet.

Jetzt machst du die Datei ausführbar und startest sie:

```
chmod +x hello.sh
./hello.sh
# Willkommen im Linux-Kurs!
```

`chmod +x` gibt der Datei das Recht zum Ausführen. Ohne dieses Recht kommt `Permission denied`. Mehr über Rechte lernst du an Tag 8.

`./` bedeutet: in diesem Ordner. Ohne `./` sucht die Shell nur in den Systemordnern wie `/usr/bin`. Dort liegt dein Skript nicht. Dann kommt `command not found`.

Achtung: Vergiss `chmod +x` nicht. Diesen Fehler machen alle beim ersten Mal.

Ausgaben und Pausen

`echo` gibt Text aus. `sleep` wartet.

```
#!/bin/bash
# Zeigt drei Schritte mit Pause
echo "Schritt 1: Ordner anlegen"
sleep 2
echo "Schritt 2: Datei anlegen"
sleep 2
echo "Fertig."
```

`sleep 2` wartet zwei Sekunden. So kann ein Mensch mitlesen, was das Skript gerade tut.

Tipp: In der Klausur wird beides verlangt: vor jedem Schritt ein `echo`, nach jedem Schritt ein `sleep 2`.

Häufige Fehler beim Starten

- **Permission denied**. Das Skript ist nicht ausführbar. Lösung: `chmod +x skript.sh`.
- **command not found** **beim Start**. Das `./` fehlt. Lösung: `./skript.sh`.
- **bad interpreter**. Die erste Zeile hat einen Tippfehler, zum Beispiel `#!/bin/bsh`. Richtig ist `#!/bin/bash`.
- **Das Skript macht nichts Neues**. Du hast in nano nicht gespeichert. Erst Strg+O und Enter, dann Strg+X.
- **cd im Skript wirkt nicht**. Ein `cd` im Skript gilt nur im Skript. Danach ist dein Terminal noch im alten Ordner.

Aufgaben · Teil 1

Dein erstes eigenes Skript

Wozu? Ein Skript merkt sich Befehle für dich. Du schreibst sie einmal. Danach startest du sie, so oft du willst.

1. Lege den Ordner an: `mkdir -p ~/linux-kurs/tag04` . Wenn es den Ordner schon gibt, passiert nichts.
2. Wechsle hinein: `cd ~/linux-kurs/tag04`
3. Öffne eine neue Datei: `nano ueber-mich.sh`
4. Tippe in die erste Zeile: `#!/bin/bash`
5. Tippe in die nächste Zeile einen Kommentar: `# Zeigt drei Saetze ueber mich`
6. Tippe in die nächste Zeile: `echo "Ich bin Carmen."` Schreibe deinen Namen statt Carmen.
7. Tippe in die nächste Zeile: `echo "Mein Lieblingsbefehl ist ls -la."` Schreibe deinen Lieblingsbefehl statt `ls -la`.
8. Tippe in die nächste Zeile: `echo "Mein Lieblingsessen ist Pizza."` Schreibe dein Lieblingsessen statt Pizza.
9. Speichere mit Strg+O und Enter. Beende mit Strg+X.
10. Mache das Skript ausführbar: `chmod +x ueber-mich.sh`
11. Starte das Skript: `./ueber-mich.sh`
12. Prüfe die Rechte: `ls -l ueber-mich.sh` . Jetzt steht in den Rechten ein `x` , zum Beispiel `-rwxrwxr-x` .

So sieht der Start aus:

```
Ich bin Carmen.  
Mein Lieblingsbefehl ist ls -la.  
Mein Lieblingsessen ist Pizza.
```

✓ Bestätigt

Arbeitsplatz per Skript

Wozu? Neue Kollegen brauchen immer dieselben Ordner. Von Hand dauert das Minuten, und man vergisst leicht etwas. Ein Skript macht es in Sekunden und immer gleich.

Das Skript hat 3 Schritte. Ein Schritt hat drei Zeilen: erst `echo`, dann der Befehl, dann `sleep 2`. So will es auch die Klausur.

1. Wechsle in den Kursordner: `cd ~/linux-kurs/tag04`
2. Öffne eine neue Datei: `nano arbeitsplatz.sh`
3. Tippe den Anfang und Schritt 1:

```
#!/bin/bash
# Legt einen Arbeitsplatz fuer neue Kollegen an
echo "Schritt 1: Ordner anlegen"
mkdir -p arbeitsplatz/dokumente arbeitsplatz/skripte
sleep 2
```

4. Tippe darunter Schritt 2:

```
echo "Schritt 2: Datei anlegen"
touch arbeitsplatz/dokumente/notizen.txt
sleep 2
```

5. Tippe darunter Schritt 3. Nach dem letzten Schritt brauchst du kein `sleep`.

```
echo "Schritt 3: Ergebnis zeigen"
ls -R arbeitsplatz
```

6. Speichere mit Strg+O und Enter. Beende mit Strg+X.
7. Mache das Skript ausführbar: `chmod +x arbeitsplatz.sh`
8. Starte das Skript: `./arbeitsplatz.sh`

So beginnt die Ausgabe:

```
Schritt 1: Ordner anlegen
Schritt 2: Datei anlegen
```

✓ Bestätigt

Der Steckbrief als Skript

Wozu? Ein Admin will oft schnell wissen: Wie heißt der Rechner? Mit welchem Benutzer arbeite ich? Wie voll ist die Festplatte? Ein Skript zeigt alles mit einem Befehl.

Das Skript zeigt 4 Angaben. Vor jeder Angabe steht eine Überschrift mit `echo`.

1. Wechsle in den Kursordner: `cd ~/linux-kurs/tag04`
2. Öffne eine neue Datei: `nano steckbrief.sh`
3. Tippe diese Zeilen:

```
#!/bin/bash
# Zeigt einen Steckbrief des Rechners
echo "Rechner:"
cat /etc/hostname
echo "Benutzer:"
whoami
```

4. Tippe in die nächste Zeile: `echo "Festplatte:"`
5. Tippe in die nächste Zeile: `df -h /`
6. Tippe in die nächste Zeile: `echo "Eintraege in /etc:"`
7. Tippe in die nächste Zeile: `ls /etc | wc -l`
8. Speichere mit Strg+O und Enter. Beende mit Strg+X.
9. Mache das Skript ausführbar: `chmod +x steckbrief.sh`
10. Starte das Skript: `./steckbrief.sh`
11. Schreibe die Ausgabe in eine Datei: `./steckbrief.sh > steckbrief.txt`
12. Prüfe die Datei: `cat steckbrief.txt`

So beginnt die Ausgabe:

```
Rechner:
lubuntu
```

✓ Bestätigt

Quiz · Teil 1

Das Quiz gibt keine Punkte. Du kannst es so oft wiederholen, wie du willst.

Quiz abgegeben. Ergebnis: **4 / 4 richtig**

1. Was steht in der ersten Zeile eines Bash-Skripts?

Deine Antwort: `#!/bin/bash`

Richtig

Erklärung:

Diese Zeile heißt Shebang. Sie sagt dem System, welche Shell das Skript ausführt.

2. Welcher Befehl macht die Datei `hello.sh` ausführbar?

Deine Antwort: `chmod +x hello.sh`

Richtig

Erklärung:

`chmod +x` setzt das Ausführungsrecht. Ohne das Recht kommt `Permission denied`.

3. Warum startest du ein Skript mit `./hello.sh` und nicht nur mit `hello.sh` ?

`./` macht das Skript ausführbar

Ohne `./` sucht die Shell nur in den Systemordnern und findet das Skript nicht (Deine Antwort)

`./` lädt das Skript in die Bash

Skripte müssen immer mit einem Punkt anfangen

Richtig

Erklärung:

`./` heißt "in diesem Ordner". Ohne `./` kommt `command not found`.

4. Was macht `sleep 3` in einem Skript?

Es beendet das Skript nach 3 Sekunden

Es wartet 3 Sekunden und macht dann weiter (Deine Antwort)

Es fährt den Rechner in 3 Minuten herunter

Es wiederholt den letzten Befehl 3 mal

Richtig

Erklärung:

`sleep` wartet die angegebene Zahl an Sekunden. So kann ein Mensch die Ausgabe mitlesen.

Teil 2: Variablen und Eingaben nach der großen Pause

Variablen

Eine Variable merkt sich etwas für dich. Zum Beispiel einen Namen, einen Ordner oder ein Datum.

So legst du eine Variable an:

```
NAME="Carmen"
```

Links steht der Name der Variable: `NAME` . Rechts steht, was sie sich merkt: `Carmen` .

Kein Leerzeichen um das Gleichheitszeichen. `NAME = "Carmen"` ist falsch.

Später brauchst du den Inhalt. Dann schreibst du ein `$` vor den Namen. Die Bash setzt dort ein, was die Variable sich gemerkt hat:

```
#!/bin/bash
NAME="Carmen"
echo "Hallo $NAME"
# Hallo Carmen
```

Wozu das? Ein Skript braucht oft mehrmals denselben Ordner. Mit einer Variable schreibst du ihn nur einmal:

```
ORDNER="/home/carlos/linux-kurs"
mkdir -p $ORDNER
ls $ORDNER
```

Ändert sich der Ordner, änderst du nur die erste Zeile.

Große Buchstaben sind keine Pflicht. Es ist nur eine Gewohnheit. So sieht man Variablen sofort. Linux unterscheidet aber groß und klein: `NAME` und `name` sind zwei verschiedene Variablen.

Variablen vom System

Manche Variablen gibt es schon. Das System legt sie für dich an:

Variable	Was drin steht	Beispiel
<code>\$USER</code>	dein Benutzername	<code>carlos</code>
<code>\$HOME</code>	dein Home-Verzeichnis	<code>/home/carlos</code>

Variable	Was drin steht	Beispiel
\$PWD	der Ordner, in dem du gerade bist	/home/carlos/linux-kurs
\$HOSTNAME	der Name des Rechners	lubuntu

```
echo "Ich bin $USER und wohne in $HOME"  
# Ich bin carlos und wohne in /home/carlos
```

Die Ausgabe eines Befehls speichern

Du kannst die Ausgabe eines Befehls in einer Variable speichern. Dafür schreibst du den Befehl in `$( )`:

```
HEUTE=$(date +%d.%m.%Y)  
echo "Heute ist der $HEUTE"  
# Heute ist der 24.09.2026
```

Hinter dem `+` bestimmst du die Form des Datums:

Zeichen	Bedeutung	Beispiel
%d	Tag	24
%m	Monat	09
%Y	Jahr	2026
%H:%M	Stunde und Minute	10:15

Für Dateinamen nimmst du besser `date +%Y-%m-%d`. Das gibt `2026-09-24`. Dann sortiert `ls` die Dateien richtig nach Datum. Diese Form ist eine internationale Norm. Sie heißt ISO 8601 und gilt seit 1988.

Eingabe vom Benutzer

`read` hält das Skript an. Es wartet, bis jemand etwas tippt und Enter drückt. Die Eingabe landet in einer Variable.

```
#!/bin/bash
# Fragt nach dem Namen und gruesst
echo "Wie heisst du?"
read NAME
echo "Hallo $NAME!"
```

Kürzer geht es mit `-p` . Dann steht die Frage in derselben Zeile wie die Antwort:

```
read -p "Wie heisst du? " NAME
echo "Hallo $NAME!"
```

Häufige Fehler bei Variablen

- **NAME: command not found** . Leerzeichen um das `=` . Richtig ist `NAME="Carmen"` .
- `$NAME="Carmen"` . Beim Anlegen kommt kein `$` davor. Das `$` brauchst du nur beim Lesen.
- **Die Variable ist leer**. Oft ein Tippfehler bei groß und klein. `$name` ist nicht `$NAME` .
- `sh skript.sh` **statt** `./skript.sh` . `sh` startet die Shell Dash, nicht Bash. Dann ist zum Beispiel `$HOSTNAME` leer. Starte Skripte mit `./skript.sh` .

Aufgaben · Teil 2

Backup mit Datum

Wozu? Ein Backup braucht das Datum im Namen. Sonst überschreibt das neue Backup das alte. Mit einer Variable schreibst du das Datum nur einmal und nutzt es überall.

1. Wechsle in den Kursordner: `cd ~/linux-kurs/tag04`
2. Lege eine Datei zum Sichern an: `echo "Meine wichtige Notiz" > notiz.txt`
3. Öffne eine neue Datei: `nano backup.sh`
4. Tippe diese 4 Zeilen. Die zwei Variablen merken sich den Zielordner und das Datum.

```
#!/bin/bash
# Sichert notiz.txt mit dem Datum im Namen
ZIEL="backup"
HEUTE=$(date +%Y-%m-%d)
```

5. Tippe darunter den Schritt für den Ordner:

```
echo "Lege den Ordner $ZIEL an"
mkdir -p $ZIEL
```

6. Tippe darunter den Schritt für die Kopie:

```
echo "Kopiere notiz.txt"
cp notiz.txt $ZIEL/notiz-$HEUTE.txt
```

7. Tippe darunter den Schritt für die Anzeige:

```
echo "Inhalt von $ZIEL:"
ls -l $ZIEL
```

8. Speichere mit Strg+O und Enter. Beende mit Strg+X.
9. Mache das Skript ausführbar: `chmod +x backup.sh`
10. Starte das Skript: `./backup.sh`

So beginnt die Ausgabe:

Lege den Ordner backup an
Kopiere notiz.txt

Am Ende steht die Kopie mit deinem heutigen Datum, zum Beispiel `notiz-2026-09-24.txt` .

✓ **Bestätigt**

Ein Skript, das fragt

Wozu? Manche Skripte brauchen eine Angabe von einem Menschen. Zum Beispiel den Namen für einen neuen Benutzer. Das machst du an Tag 7. Heute übst du das Fragen.

1. Wechsle in den Kursordner: `cd ~/linux-kurs/tag04`
2. Öffne eine neue Datei: `nano Lieblingsbefehl.sh`
3. Tippe diese 2 Zeilen:

```
#!/bin/bash  
# Fragt nach Name und Lieblingsbefehl
```

4. Tippe in die nächste Zeile. Sie fragt nach dem Namen: `read -p "Wie heisst du? " NAME`
5. Tippe in die nächste Zeile. Sie fragt nach dem Lieblingsbefehl: `read -p "Was ist dein Lieblingsbefehl? " BEFEHL`
6. Tippe in die nächste Zeile. Sie speichert das Datum: `HEUTE=$(date +%d.%m.%Y)`
7. Tippe darunter diese 2 Zeilen. Sie zeigen die Antworten:

```
echo "Hallo $NAME, heute ist der $HEUTE."  
echo "Dein Lieblingsbefehl ist $BEFEHL."
```

8. Tippe darunter diese 2 Zeilen. Sie schreiben dasselbe in die Datei `antworten.txt`. Die erste Zeile legt die Datei mit `>` an. Die zweite hängt mit `>>` an.

```
echo "Hallo $NAME, heute ist der $HEUTE." > antworten.txt  
echo "Dein Lieblingsbefehl ist $BEFEHL." >> antworten.txt
```

9. Tippe in die letzte Zeile: `cat antworten.txt`. Diese Zeile zeigt die Datei am Ende an.
10. Speichere mit Strg+O und Enter. Beende mit Strg+X.
11. Mache das Skript ausführbar: `chmod +x Lieblingsbefehl.sh`
12. Starte das Skript: `./Lieblingsbefehl.sh`. Beantworte die zwei Fragen.

So beginnt es:

```
Wie heisst du? Javier  
Was ist dein Lieblingsbefehl? ls -la
```

Hallo Javier, heute ist der 24.09.2026.

✓ Bestätigt

Quiz · Teil 2

Das Quiz gibt keine Punkte. Du kannst es so oft wiederholen, wie du willst.

Quiz abgegeben. Ergebnis: 4 / 4 richtig

1. Welche Zeile legt eine Variable richtig an?

NAME = "Carmen"

NAME="Carmen" (Deine Antwort)

\$NAME = "Carmen"

set NAME "Carmen"

Richtig

Erklärung:

Um das Gleichheitszeichen darf kein Leerzeichen stehen. Das `$` brauchst du nur beim Auslesen.

2. Wie gibst du den Inhalt der Variable `NAME` auf dem Bildschirm aus?

Deine Antwort: `echo $NAME`

Richtig

Erklärung:

Beim Auslesen kommt ein `$` vor den Namen. `echo $NAME` gibt den Wert aus, nicht das Wort `NAME`.

3. Was macht `read NAME` in einem Skript?

Es liest die Datei `NAME` ein

Es wartet auf eine Eingabe und speichert sie in der Variable `NAME` (Deine Antwort)

Es gibt den Inhalt von `NAME` aus

Es löscht die Variable `NAME`

Richtig

Erklärung:

`read` hält das Skript an und wartet auf Enter. Mit `read -p "Frage: " NAME` kommt die Frage gleich mit.

4. Welche Zeile speichert das heutige Datum in der Variable `HEUTE` ?

`HEUTE=date`

`HEUTE=$(date +%d.%m.%Y)` (Deine Antwort)

`HEUTE = $(date +%d.%m.%Y)`

`$HEUTE=(date +%d.%m.%Y)`

Richtig

Erklärung:

`$()` führt den Befehl aus und gibt seine Ausgabe weiter. `HEUTE=date` speichert nur das Wort `date`. Um das `=` darf kein Leerzeichen stehen, und beim Anlegen kommt kein `$` davor.

Bonusaufgaben

Freiwillig, für alle, die früher fertig sind. Bonusaufgaben zählen nicht für die Tagesmedaille. Sind alle Bonusaufgaben eines Tages bestätigt, gibt es eine Bonusmedaille.

Einfache oder doppelte Anführungszeichen Bonus

Wozu? In Skripten stehen oft Anführungszeichen. Es gibt zwei Sorten. Sie machen etwas Verschiedenes. Wenn du den Unterschied kennst, findest du Fehler schneller.

1. Tippe im Terminal: `echo "Ich bin $USER"`
2. Tippe im Terminal: `echo 'Ich bin $USER'`
3. Tippe im Terminal: `echo "Heute ist $(date +%A)"`
4. Tippe im Terminal: `echo 'Heute ist $(date +%A)'`
5. Vergleiche die Ausgaben. Was machen die einfachen Anführungszeichen `' '` anders?
6. Öffne eine neue Datei: `nano ~/linux-kurs/tag04/anfuehrung.txt` . Schreibe deine Antwort in zwei Sätzen hinein. Speichere und beende.
7. Prüfe mit `cat ~/linux-kurs/tag04/anfuehrung.txt` .

✓ Bestätigt

Rechnen im Skript Bonus

Wozu? Skripte müssen oft rechnen. Zum Beispiel: Wie viele Tage noch bis zur Klausur? Wie viel Platz ist noch frei?

Neu ist `$(( ))`. Damit rechnet die Bash mit ganzen Zahlen. Beispiel: `echo $(( 3 + 4 ))` gibt 7.

1. Rechne im Terminal: `echo $(( 3 + 4 ))`
2. Rechne im Terminal: `echo $(( 6 * 7 ))`
3. Rechne im Terminal: `echo $(( 7 / 2 ))`. Das Ergebnis ist 3 und nicht 3.5. Warum? Merke dir deine Antwort.
4. Wechsle in den Kursordner: `cd ~/linux-kurs/tag04`
5. Öffne eine neue Datei: `nano alter.sh`
6. Tippe diese Zeilen:

```
#!/bin/bash
# Rechnet aus, wie alt ich dieses Jahr werde
read -p "In welchem Jahr bist du geboren? " JAHR
JETZT=$(date +%Y)
```

7. Tippe in die nächste Zeile. Sie rechnet: `ALTER=$(( JETZT - JAHR ))`
8. Tippe in die nächste Zeile: `echo "Du wirst dieses Jahr $ALTER."`
9. Tippe in die letzte Zeile deine Antwort aus Schritt 3. Schreibe sie als Kommentar, also mit `#` davor.
10. Speichere mit Strg+O und Enter. Beende mit Strg+X.
11. Mache das Skript ausführbar: `chmod +x alter.sh`
12. Starte das Skript: `./alter.sh`

✓ Bestätigt